

CSE327

Software Engineering

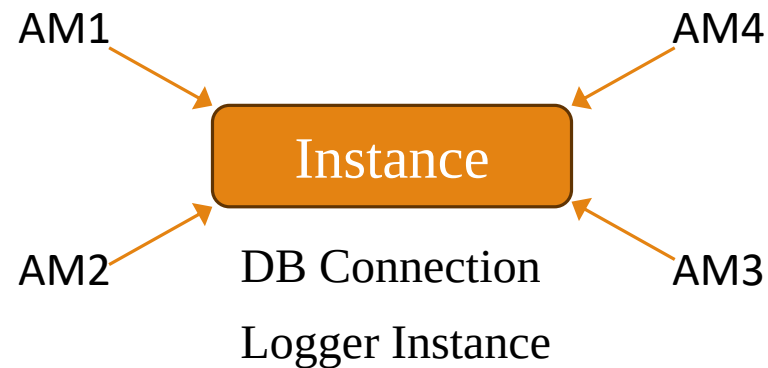
AKM IQTIDAR NEWAZ [IQN]

LECTURE - 11

Singleton Design Pattern

- ❑ When only one instance of a class is needed (shared resource) across your application, you have to allow the access to that instance from the whole application but also we don't need a new instance every time. we only need that one instance which could be utilized across the whole application

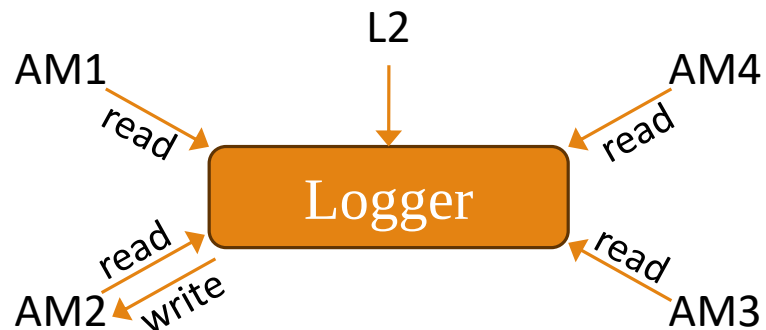
AM = App Module



An instance of a class
or shared resource

Singleton Design Pattern

- ❑ One can argue that we can solve this problem just by saving that instance in a global variable and allow access from different application on module / different programs in the same application to that global variable. Let's say we do not.
- ❑ We save the logger instance in the global variable. Allow access from different application module to read that variable. It is fine as long as different application module are just reading that variable but what if one particular module overwrites it. In that case this global variable have changes and here is the problem.

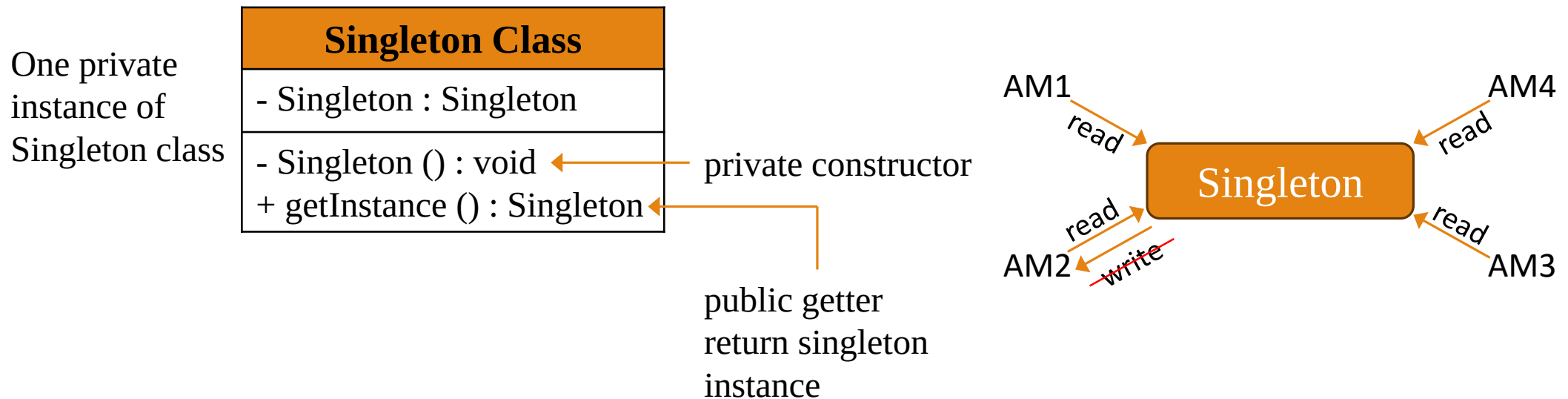


global variable logger;
logger = L1;
logger = L2 (AM2)

modified that's why we
can't use global variable

Singleton Design Pattern

Solution: Try to initialize that variable using a private constructor and we don't allow anyone else to access that constructor. You only allow to access that variable by getter.



Remember: Singleton class never accepts parameters. If it accepts parameters, then it becomes a factory design pattern. So avoid it.

Eager Loading & Lazy Loading

❑ Eager Loading

- ❑ The instance is already initialized as soon as the application is up. Application starts and instance is already initialized.
- ❑ If nobody needs it, it will be there and utilize memory.
- ❑ Works well with one singleton class.

❑ Lazy Loading

- ❑ Whenever our application starts the singleton instance is not initialized, it is initialized only when someone invokes it / tries to read it.
- ❑ We don't initialize it. When application module needs it, then it creates the instance. If its available, no need to create it. Optimization is done.
- ❑ Suitable for multiple singleton class

Example: Logger

```
#pragma once
#include <iostream>
using namespace std;

class Logger
{
public:
    static Logger *getLogger();
    void Log(string msg);

private:
    static int counter;
    Logger();
    static Logger
    *loggerInstance;
};
```

logger.h

```
#include "logger.h"

int Logger::counter = 0;
Logger *Logger::loggerInstance = nullptr;

Logger ::Logger()
{
    counter++;
    cout << "New instance of Logger created. Number of instances created: " << counter << endl;
}

void Logger ::Log(string msg)
{
    cout << msg << endl;
}

Logger *Logger ::getLogger()
{
    if (loggerInstance == nullptr)
    {
        loggerInstance = new Logger();
    }
    return loggerInstance;
}
```

logger.cpp

Example: Logger

```
#include "logger.h"

int main()
{
    Logger *logger1 = Logger ::getLogger();
    logger1->Log("this msg is from user 1");
    cout << endl;
    Logger *logger2 = Logger ::getLogger();
    logger2->Log("this msg is from user 2");
}
```

notThread.cpp

Output:

New instance of Logger created. Number of instances created: 1
this msg is from user 1

this msg is from user 2

Example: Logger (Not Thread Safe)

```
#include "logger.h"
#include <thread>

void userLogs()
{
    Logger *logger1 = Logger ::getLogger();
    logger1->Log("this msg is from user 1");
    cout << endl;
}

void user2Logs()
{
    Logger *logger2 = Logger ::getLogger();
    logger2->Log("this msg is from user 2");
}

int main()
{
    thread t1(userLogs);
    thread t2(user2Logs);
    t1.join();
    t2.join();
}
```

threadMain.cpp

Output:

New instance of Logger created. Number of instances created: 2
this msg is from user 1

New instance of Logger created. Number of instances created: 2
this msg is from user 2

Example: Logger (Thread Safe)

```
#pragma once
#include <iostream>
#include <mutex>
using namespace std;

class Logger
{
public:
    static Logger *getLogger();
    void Log(string msg);

private:
    static int counter;
    static mutex mtx;
    Logger();
    static Logger *loggerInstance;
};
```

threadSafellogger.h

```
#include "threadSafellogger.h"

int Logger::counter = 0;
Logger *Logger::loggerInstance = nullptr;
mutex Logger::mtx;

Logger ::Logger()
{
    counter++;
    cout << "New instance of Logger created. Number of instances created: " << counter << endl;
}

void Logger ::Log(string msg)
{
    cout << msg << endl;
}

Logger *Logger ::getLogger()
{
    mtx.lock();
    if (loggerInstance == nullptr)
    {
        loggerInstance = new Logger();
    }
    mtx.unlock();
    return loggerInstance;
}
```

threadSafeLogger.cpp

Example: Logger (Thread Safe)

```
#include "threadSafelogger.h"
#include <thread>

void userLogs()
{
    Logger *logger1 = Logger ::getLogger();
    logger1->Log("this msg is from user 1");
    cout << endl;
}

void user2Logs()
{
    Logger *logger2 = Logger ::getLogger();
    logger2->Log("this msg is from user 2");
}

int main()
{
    thread t1(userLogs);
    thread t2(user2Logs);
    t1.join();
    t2.join();
}
```

Output:

New instance of Logger created. Number of instances created: 1

this msg is from user 1

this msg is from user 2